

Capstone Project Refresher Guide

Quick Reference for Key Concepts from Modules 1-3

This guide provides a quick refresher on essential concepts you'll need for your capstone project. Use it as a reference when you need to remember syntax or approaches from earlier in the program.

Python Fundamentals

 **Covered in: Module 1, Weeks 1-5**

Data Structures

```
# Lists - ordered, mutable collections
data = [1, 2, 3, 4, 5]
data.append(6)           # Add to end
data[0]                  # Access by index

# Dictionaries - key-value pairs
person = {'name': 'Alice', 'age': 30, 'city': 'NYC'}
person['name']            # Access by key
person['email'] = 'a@example.com' # Add new key

# List comprehensions - concise filtering/transformation
squares = [x**2 for x in range(10)]
evens = [x for x in data if x % 2 == 0]
```

Loops and Conditionals

```
# For Loops
for item in data:
    print(item)

# Conditionals
if value > 100:
    category = 'high'
elif value > 50:
    category = 'medium'
else:
    category = 'low'
```

Functions

```
def calculate_total(prices, tax_rate=0.08):
    """Calculate total with tax"""
    subtotal = sum(prices)
```

```
return subtotal * (1 + tax_rate)
```

Usage

```
total = calculate_total([10, 20, 30], tax_rate=0.10)
```

Pandas Essentials



Covered in: Module 2, Weeks 1-2

Loading and Inspecting Data

```
import pandas as pd
```

Load data

```
df = pd.read_csv('data.csv')
```

```
df = pd.read_csv('data.csv', encoding='utf-8') # If encoding issues
```

Quick inspection

```
df.head() # First 5 rows
```

```
df.info() # Column types, non-null counts
```

```
df.describe() # Summary statistics
```

```
df.shape # (rows, columns)
```

```
df.columns # Column names
```

Data Cleaning

Handle missing values

```
df.isnull().sum() # Count missing per column
```

```
df.dropna() # Remove rows with any missing
```

```
df.dropna(subset=['col']) # Remove if specific column missing
```

```
df['col'].fillna(df['col'].mean()) # Fill with mean
```

```
df['col'].fillna('Unknown') # Fill with placeholder
```

Remove duplicates

```
df.duplicated().sum() # Count duplicates
```

```
df.drop_duplicates() # Remove duplicates
```

```
df.drop_duplicates(subset=['email']) # Based on specific column
```

Fix data types

```
df['date'] = pd.to_datetime(df['date'])
```

```
df['price'] = pd.to_numeric(df['price'], errors='coerce')
```

```
df['category'] = df['category'].astype('category')
```

Clean column names

```
df.columns = df.columns.str.lower().str.replace(' ', '_')
```

Filtering and Selecting

```
# Select columns
df['name']           # Single column (Series)
df[['name', 'age']]  # Multiple columns (DataFrame)

# Filter rows
df[df['age'] > 30]     # Simple condition
df[df['city'].isin(['NYC', 'LA'])] # Multiple values
df[(df['age'] > 25) & (df['salary'] > 50000)] # Multiple conditions

# Sort
df.sort_values('age') # Ascending
df.sort_values('age', ascending=False) # Descending
df.sort_values(['city', 'age']) # Multiple columns
```

Grouping and Aggregation

```
# Basic groupby
df.groupby('category')['price'].mean()
df.groupby('city').size() # Count per group

# Multiple aggregations
df.groupby('category').agg({
    'price': ['mean', 'sum', 'count'],
    'quantity': 'sum'
})

# Add group statistics back to DataFrame
df['city_avg_price'] = df.groupby('city')['price'].transform('mean')
```

Combining DataFrames

```
# Concatenate (stack)
combined = pd.concat([df1, df2]) # Stack vertically
combined = pd.concat([df1, df2], axis=1) # Side by side

# Merge (join)
merged = pd.merge(df1, df2, on='id') # Inner join
merged = pd.merge(df1, df2, on='id', how='left') # Left join
merged = pd.merge(df1, df2, left_on='emp_id', right_on='employee_id')
```

Data Visualization

 **Covered in: Module 2, Weeks 2 & 4**

Matplotlib Basics

```
import matplotlib.pyplot as plt
```

```
# Line plot
```

```
plt.plot(x, y)
plt.xlabel('X Label')
plt.ylabel('Y Label')
plt.title('My Plot')
plt.show()
```

```
# Bar chart
```

```
plt.bar(categories, values)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# Scatter plot
```

```
plt.scatter(x, y, alpha=0.5)
plt.show()
```

```
# Histogram
```

```
plt.hist(data, bins=20)
plt.show()
```

Pandas Plotting

```
# Quick plots from DataFrames
```

```
df['column'].plot(kind='hist')
df.plot(x='date', y='value', kind='line')
df.groupby('category')['value'].sum().plot(kind='bar')
```

SQL Essentials

 **Covered in: Module 3, Weeks 1-4**

Basic Queries

```
-- Select specific columns
```

```
SELECT name, age, city
FROM customers
WHERE age > 30
ORDER BY age DESC
LIMIT 10;
```

```
-- Aggregate functions
```

```
SELECT
    category,
    COUNT(*) as count,
```

```

        AVG(price) as avg_price,
        SUM(quantity) as total_quantity
FROM products
GROUP BY category
HAVING COUNT(*) > 5
ORDER BY avg_price DESC;

```

Joins

-- Inner join (only matching rows)

```

SELECT
    e.name,
    d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;

```

-- Left join (all from left table)

```

SELECT
    c.name,
    o.order_date,
    o.total
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;

```

-- Multiple joins

```

SELECT
    o.order_id,
    c.name as customer,
    p.product_name,
    oi.quantity
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id;

```

Subqueries and CTEs

-- Subquery

```

SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

```

-- CTE (Common Table Expression)

```

WITH high_value_orders AS (
    SELECT customer_id, SUM(total) as total_spent
    FROM orders
    WHERE status = 'completed'
    GROUP BY customer_id
    HAVING SUM(total) > 1000
)
SELECT

```

```
        c.name,  
        h.total_spent  
FROM customers c  
INNER JOIN high_value_orders h ON c.id = h.customer_id;
```

Working with APIs

 Covered in: Module 2, Week 3

Making API Requests

```
import requests  
import json  
  
# GET request  
response = requests.get('https://api.example.com/data')  
data = response.json()  
  
# With parameters  
params = {'city': 'Louisville', 'units': 'imperial'}  
response = requests.get('https://api.example.com/weather', params=params)  
  
# With API key  
headers = {'Authorization': 'Bearer YOUR_API_KEY'}  
response = requests.get('https://api.example.com/data', headers=headers)  
  
# Check status  
if response.status_code == 200:  
    data = response.json()  
else:  
    print(f"Error: {response.status_code}")
```

Converting API Data to DataFrame

```
# From JSON response  
data = response.json()  
df = pd.DataFrame(data)  
  
# From nested JSON  
df = pd.json_normalize(data, record_path=['items'])
```

File Operations

 Covered in: Module 2, Weeks 1-3

Reading Files

```
# CSV
df = pd.read_csv('data.csv')

# JSON
df = pd.read_json('data.json')
with open('data.json', 'r') as f:
    data = json.load(f)

# Excel
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

Writing Files

```
# CSV
df.to_csv('output.csv', index=False)

# JSON
df.to_json('output.json', orient='records')
with open('output.json', 'w') as f:
    json.dump(data, f, indent=2)

# Excel
df.to_excel('output.xlsx', index=False, sheet_name='Results')
```

Common Patterns for Capstone



Integrates concepts from all modules

Data Pipeline Structure

```
# 1. Load data
df = pd.read_csv('raw_data.csv')

# 2. Clean data
df = df.dropna(subset=['important_column'])
df['date'] = pd.to_datetime(df['date'])
df.columns = df.columns.str.lower().str.replace(' ', '_')

# 3. Transform/analyze
summary = df.groupby('category').agg({
    'value': ['mean', 'sum', 'count']
})

# 4. Visualize
summary.plot(kind='bar')
plt.title('Summary by Category')
```

```
plt.tight_layout()
plt.savefig('summary_chart.png')

# 5. Export results
summary.to_csv('summary_results.csv')
```

SQL to Pandas Workflow

```
import sqlite3

# Connect to database
conn = sqlite3.connect('database.db')

# Query to DataFrame
query = """
    SELECT category, AVG(price) as avg_price
    FROM products
    GROUP BY category
    """
df = pd.read_sql(query, conn)

# Work with DataFrame
df_filtered = df[df['avg_price'] > 50]

# Close connection
conn.close()
```

Debugging Tips

Common Issues and Fixes

KeyError: 'column_name'

```
# Check what columns exist
print(df.columns.tolist())
# Clean column names
df.columns = df.columns.str.strip().str.lower()
```

ValueError: could not convert string to float

```
# Use errors='coerce' to handle invalid values
df['price'] = pd.to_numeric(df['price'], errors='coerce')
```

FileNotFoundError

```
import os
# Check current directory
print(os.getcwd())
```



```
# List files
print(os.listdir('.'))
```

SQL Syntax Error

```
-- Check for:
-- - Missing commas between columns
-- - Quotes around text values ('text' not text)
-- - Semicolon at end
-- - Correct table/column names
```

Capstone Project Requirements

Version Control (Git/GitHub)

```
# Initialize repository
git init
git add .
git commit -m "Initial commit"

# Regular commits (aim for 10+ throughout project)
git add filename.py
git commit -m "Add data cleaning function"
git push origin main

# Check commit history
git log --oneline
```

Best practices:

- Commit frequently (after completing each logical step)
- Write clear commit messages describing what changed
- Use command line for commits (not file uploads)
- Space commits across your project timeline

Example commit progression for capstone:

1. Initial commit: Add README and project structure
2. Add raw datasets and data source documentation
3. Create initial data loading notebook
4. Add data cleaning functions for missing values
5. Implement duplicate removal and data type fixes
6. Add exploratory data analysis with summary statistics
7. Create database schema and ERD diagram
8. Build SQLite database with initial tables
9. Add SQL queries for data analysis

10. Create visualization functions for key insights
11. Add bar charts and line plots for trends
12. Implement custom analysis functions
13. Add requirements.txt and update README
14. Final documentation and code cleanup

Project Structure

```
capstone-project/
├── README.md           # Project overview and instructions
├── requirements.txt     # Python dependencies
├── data/               # Raw data files
│   ├── dataset1.csv
│   └── dataset2.csv
├── notebooks/         # Jupyter notebooks
│   └── analysis.ipynb
├── src/               # Python scripts (optional)
│   └── functions.py
├── output/            # Generated files
│   ├── cleaned_data.csv
│   └── visualizations/
```

Use relative paths:

 *Good - works on any computer*

```
df = pd.read_csv('data/dataset.csv')
```

 *Bad - only works on your computer*

```
df = pd.read_csv('/Users/yourname/Desktop/project/data/dataset.csv')
```

README.md Template

Project Title

Brief description of your project and what questions you're answering.

Data Sources

- [Dataset 1 Name](link-to-source) - Description
- [Dataset 2 Name](link-to-source) - Description

Setup Instructions

1. Clone the repository:

```
``bash
git clone https://github.com/yourusername/project-name.git
cd project-name
``
```

1. Create virtual environment:

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Run the analysis:

```
jupyter notebook notebooks/analysis.ipynb
```

Key Findings

- Finding 1
- Finding 2
- Finding 3

Technologies Used

- Python 3.x
- Pandas
- Matplotlib
- SQLite3

```
### Creating requirements.txt
```bash
After installing all packages, generate requirements.txt
pip freeze > requirements.txt
```

```
Or manually create with key packages:
pandas==2.0.0
matplotlib==3.7.0
requests==2.31.0
```

## Virtual Environments

```
Create virtual environment
python -m venv venv
```

```
Activate (Mac/Linux)
source venv/bin/activate
```

```
Activate (Windows)
venv\Scripts\activate
```

```
Install packages
pip install pandas matplotlib
```

*# Deactivate when done*  
deactivate

## Database & ERD Requirements

- Design a relational schema (multiple related tables)
- Create an ERD (Entity Relationship Diagram) showing table relationships
- Build database in SQLite3
- Write at least 3 intermediate SQL queries (joins, subqueries, aggregations)

### ERD Tools (from Module 3):

- [LucidChart](#) - Free tier available, used in course tutorials
  - [Draw.io / diagrams.net](#) - Free, web or desktop
- 

## Best Practices for Capstone

1. **Document your process** - Use markdown cells to explain your decisions
2. **Keep code organized** - One logical step per cell
3. **Test incrementally** - Don't write everything at once
4. **Handle errors gracefully** - Use try/except for API calls and file operations
5. **Create visualizations** - Charts make your analysis clearer
6. **Export results** - Save cleaned data and summary statistics
7. **Comment your code** - Explain complex logic
8. **Use meaningful variable names** - `customer_df` not `df1`

## Understanding Rubric Requirements

**“3+ EDA techniques”** means using different methods to explore your data:

- Summary statistics (`.describe()`, `.value_counts()`)
- Grouping and aggregation (`.groupby()`)
- Filtering and sorting to find patterns
- Correlation analysis
- Distribution analysis

**“Meaningfully combined datasets”** means:

- Datasets relate to the same topic/question
- You use merge/join to connect them (not just stacking)
- The combination enables analysis you couldn't do with one dataset alone

**“Purposeful custom functions”** means:

- Functions that do real work in your analysis (not just `add(a, b)`)

- Examples: data cleaning, calculations, transformations
- Each function should have a clear purpose in your project

**“3+ different chart types”** - choose charts that fit your data:

- Bar charts: comparing categories
- Line plots: trends over time
- Scatter plots: relationships between variables
- Histograms: distribution of values
- Each chart should answer a specific question

**“Markdown explains reasoning”** means:

- Why you chose this dataset
- Why you cleaned data this way
- What patterns you found and what they mean
- What conclusions you can draw

---

## Quick Reference Links

### Official Documentation

- **Pandas Documentation:** <https://pandas.pydata.org/docs/>
- **Matplotlib Gallery:** <https://matplotlib.org/stable/gallery/>
- **Python String Methods:** <https://docs.python.org/3/library/stdtypes.html#string-methods>
- **Requests Library (APIs):** <https://requests.readthedocs.io/en/latest/>

### Interactive Tutorials

- **W3Schools Python:** <https://www.w3schools.com/python/>
- **W3Schools SQL:** <https://www.w3schools.com/sql/>
- **Learn Python:** <https://www.learnpython.org/>

### Video Tutorials (Recommended)

- **Alex the Analyst - Pandas for Beginners:** <https://youtube.com/playlist?list=PLUaB-1hjhk8GZOuyLzQqLz-Qt9RIIdZZMBE>
- **Alex the Analyst - Exploratory Data Analysis:** <https://youtu.be/Liv6eeb1VfE>
- **Alex the Analyst - APIs Tutorial:** <https://youtu.be/2HfSFdPEFRg>
- **Data with Baraa - SQL Course:** [https://youtube.com/playlist?list=PLNcg\\_FV9n7qZDdRIInCl7wfOO8yZqYX5hO](https://youtube.com/playlist?list=PLNcg_FV9n7qZDdRIInCl7wfOO8yZqYX5hO)
- **Tech with Tim - Learn Pandas in 30 Minutes:** <https://youtu.be/EXlgjIBu4EU>

## Data Visualization Resources

- **The Do's and Don'ts of Data Visualization:** <https://humansofdata.atlan.com/2019/02/dos-donts-data-visualization/>
  - **Storytelling with Data (Cole Nussbaumer Knaflic):** <https://www.youtube.com/watch?v=8EMW7io4rSI>
  - **How to Choose the Best Chart:** [https://youtu.be/fBgk\\_cNCnsk](https://youtu.be/fBgk_cNCnsk)
- 

## Remember

- **Google is your friend** - Professional data analysts look things up constantly
- **Start simple** - Get basic functionality working before adding complexity
- **Test with small samples** - Use `.head()` to test on small data first
- **Read error messages** - They usually tell you exactly what's wrong
- **Ask for help** - If stuck for >30 minutes, reach out

**You've learned all these skills - this is just a reminder of what you already know!**